

AI & Machine Learning on Google Cloud

Compact study guide — important course content in plain words
Condensed from M0–M5 + AI Hypercomputer note (labs/UI click-paths and fluff removed)

Module 1 — AI foundations

Why this matters + Coffee on Wheels example

AI is moving from “make work faster” to “enable new products.” The Coffee on Wheels example shows a food-truck business using AI for location/route optimization, sales forecasting, and marketing automation. Behind the scenes: multimodal inputs (reviews, photos, street video), predictive models (sales/routes), generative models (campaigns, item suggestions), and dashboards (BigQuery + Looker). Tools mentioned: Gemini, BigQuery, Agent Platform (formerly Vertex AI), Google Maps.

Predictive AI vs generative AI

- **Predictive (discriminative) AI:** learns from history to classify or forecast (sales, churn, routes). Best for “what will happen?”
- **Generative AI:** creates new content (text, images, video, code) and can take actions via agents. Best for multimodal creation and automation.
- Often combine both: e.g. predict churn → gen-AI chatbot helps sales explore results; segment customers → gen AI writes personalized marketing.

Google Cloud AI stack (3 layers)

- **AI infrastructure:** networking/security; compute + storage (decoupled so they scale separately); data & AI products.
- **AI development:** Agent Platform end-to-end (design → deploy), foundation models, pipelines; integrates with BigQuery.
- **AI applications & solutions:** out-of-the-box tools for business users to prototype without deep coding.

Compute choices

- **Compute Engine:** most control (like a VM).
- **GKE:** containers with Kubernetes orchestration.
- **Cloud Run:** serverless convenience; Google runs infra.
- Hardware: CPUs/GPUs for general work; **TPUs** = Google’s chips optimized for ML (matrix math), faster/more energy-efficient for many AI jobs; available as Cloud TPUs.

Storage & data types

- **Unstructured** (docs, images, audio) → usually **Cloud Storage**.
- **Structured / semi-structured** → BigQuery, AlloyDB, Cloud SQL, Spanner, etc.
- BigQuery is the flagship warehouse: great for structured + JSON; can also query unstructured objects in Cloud Storage via external tables.

Data-to-AI workflow

- **Ingest & process** (batch/real-time): Pub/Sub, Dataflow, Managed Spark, Cloud Data Fusion.
- **Store & analyze:** Cloud Storage; SQL (BigQuery, AlloyDB, Cloud SQL, Spanner); NoSQL (Bigtable, Firestore); Looker for viz.
- **Activate with AI:** Agent Platform (Agent Studio, AutoML, Notebooks) for predictive and generative projects.

AI / ML vocabulary

- **AI:** umbrella term for machines mimicking intelligent behavior.
- **ML:** subset of AI — learn from data instead of hand-written rules.
- **Supervised learning:** labeled data (dog/cat photos with labels) → predict labels for new examples. Task-driven.
- **Unsupervised learning:** unlabeled data → find patterns/groups (e.g. cluster dog breeds). Data-driven.
- **Deep learning:** ML with many layers (neural nets) between input and output.
- **Generative AI:** uses foundation models / LLMs to create and interact.

BigQuery ML — build models with SQL

BigQuery ML lets analysts train models where the data already lives, using SQL instead of a separate ML stack. Typical phases:

- Extract / transform / load data into BigQuery.
- Select and preprocess features.
- Create the model inside BigQuery.
- Evaluate performance.
- Use the model for predictions.
- Gemini Code Assist can help explain, generate, and debug SQL.

Module 2 — Generative AI

What gen AI does

- Creates multimodal content: text, code, images, speech, video, even 3D.
- Driven by a **prompt** (instruction/question).
- Also takes action through **agents** (book travel, automate workflows, etc.).
- Business value: productivity, lower ops cost, new products/experiences.

Google gen AI stack

- **Foundation models:** Gemini family (general/multimodal), specialty models like Veo (video), Nano Banana (images), Embeddings.
- Gemini variants (conceptually): Pro = strongest reasoning; Flash = speed/low latency; Flash-Lite = lighter/cheaper.
- **Development:** Model Garden, Agent Studio, Agent Garden / Agent Builder, ADK.
- **Applications:** Gemini Enterprise app, Gemini Notebook (no-code entry for users).

Foundation models, multimodal, pre-train vs fine-tune

- A foundation model is large: many parameters, huge training data, heavy compute.
- **Multimodal** (Gemini): one model can take/produce text, images, video — e.g. cookie photo → recipe video walkthrough.
- Business uses: extract info from images/docs, analyze (e.g. receipt categories), Q&A, content creation.
- **Pre-trained** = general skills on massive data (horizontal AI).
- **Fine-tuned** = extra training on smaller domain data (vertical AI: healthcare, finance, retail).
- Access models via: Console UI (explore), APIs (cURL/REST), SDKs (Python/Java).

Prompt engineering & production lifecycle

- Good prompt parts: **task, context, examples**.
- Lifecycle: design prompt → evaluate/refine → build/test → integrate/deploy → monitor/optimize.
- Agent Studio helps with templates, model parameters (temperature, top-K, top-P), side-by-side comparison, code gen, Cloud Run / Cloud Shell integration.
- **Grounding / RAG:** attach your real knowledge so answers stay factual.
- Tuning spectrum: better prompts → parameter-efficient tuning → full fine-tuning.

AI agents

Agents are the next step after chatbots: they pursue goals, call tools, and run workflows. Three parts:

- **Model (brain):** reasoning / language foundation model.
- **Tools (hands/feet):** connectors to APIs, apps, data — interact with the world.
- **Orchestration (nervous system):** process, coordination, multi-step control.
- Build path by skill: Gemini Enterprise app (no-code) → Agent Studio / templates (low-code) → Agent Development Kit / ADK (pro-code).
- Model Garden: discover/fine-tune foundation models; Agent Platform: build agents at scale with governance.

Module 3 — AI development options

Same options apply to generative and predictive projects. Choose by data, expertise, time, and how much control you need.

Spectrum of options

- **No-code**: Gemini Enterprise app / for CX — out-of-box.
- **No/low-code**: Agent Platform AutoML — point-and-click models on your data.
- **Low-code**: Pre-trained APIs — ready models; no training data required.
- **Code-based**: BigQuery ML, custom training, ADK — full DIY control.

How to choose (practical guide)

- **Pre-trained APIs**: little ML expertise; common tasks (vision, language, speech); no training data; no hyperparameter tuning; fastest to use.
- **BigQuery ML**: tabular data already in BigQuery; team knows SQL; medium data size; medium flexibility.
- **Agent Platform AutoML**: own training data; want custom model with minimal coding; tabular/image/video (per product support); small–medium data; UI-driven.
- **Custom training**: need full control of architecture/frameworks; medium–large data; high ML/coding skill; longest training time; most flexible.

Agent Platform (unified AI platform)

- Solves production pain (scale, monitoring, CI/CD/CT) and ease-of-use pain (too many tools, heavy coding).
- End-to-end: prepare data → create models → deploy → manage over time.
- Supports **both** generative AI and predictive AI.
- Predictive paths: AutoML (UI) or custom training (Workbench / Colab Enterprise).
- Benefits (“4 Ss”): seamless, scalable (MLOps), sustainable (reuse artifacts/features), speedy.
- Feature store: create/share features; upload from Cloud Storage, BigQuery, or local.

AutoML — what it automates

- Goal: automate the tedious loop of features → try models → tune params → deploy.
- Phase 1 — auto preprocessing (e.g. TensorFlow Transform): numbers, datetime, text, categories, arrays, nested fields.
- Phase 2 — architecture search + tuning: neural architecture search + transfer learning (reuse pre-trained knowledge → less data/compute).
- Phase 3 — bagging ensemble of top models (often ~10, depends on budget).
- Phase 4 — prediction (online endpoint or batch).
- UI flow: create dataset → train (AutoML or custom) → get predictions.

Pre-trained APIs

- Use when you lack hundreds of thousands of labeled examples for a custom model.
- Like a wall outlet: you only need the right adapter (API + params), not the power plant (training).
- Typical call pattern: authenticate → pick model → call API → use response (SDK or REST).
- Google offers gen AI / foundation APIs, Agent Platform APIs, plus Speech, Language, Vision/Video, Document, Conversational APIs.
- Natural Language API capabilities: entities, sentiment, syntax, categories (methods like analyzeEntities, analyzeSentiment, analyzeSyntax, classifyText).
- REST: JSON request with document type/language/content → POST → parse JSON.

Custom training

- Choose when AutoML/APIs aren't flexible enough — you own architecture, frameworks, training logic.
- Runtime: **pre-built container** (Python/TF/PyTorch ready) or **custom container** (you define env/machine/disks).
- Code environments: Agent Platform Workbench (Jupyter-style) or Colab Enterprise.
- Libraries: TensorFlow, PyTorch, scikit-learn; JAX is another Google-backed high-performance option.
- tf.keras pattern: create model (layers) → compile (loss/optimizer/metrics) → fit (train for N epochs) → deploy.

Module 4 — AI development workflow

Three stages (restaurant analogy)

- **1. Data preparation** — prep ingredients: upload data + feature engineering. Data quality/quantity decide learning quality. Batch vs streaming; structured vs unstructured.
- **2. Model development** — cook & taste: iterative train ↔ evaluate.
- **3. Model serving** — serve guests: deploy + monitor. A model not in production has no business value.
- Workflow is **iterative**, not one-way (bad features → go back; drift in prod → retrain).
- Two build styles: AutoML UI (no-code) or coded pipelines (Workbench/Colab + Agent Platform Pipelines).

Data preparation details

- Sources: Cloud Storage, BigQuery, local machine.
- AutoML tabular objectives commonly: regression, classification, forecasting.
- A **feature** = input factor for prediction (column / independent variable).
- **Feature Store**: central place to manage/serve/share features (online low-latency + offline batch); also embeddings + similarity for gen AI.
- Online feature serving setup (concept): prepare data in BigQuery → (optional) register feature groups → create feature view → serve latest values.

Model development & evaluation

- Training setup typically needs: dataset, objective, AutoML vs custom, target column (supervised), feature transforms, compute budget / early stopping.
- **Confusion matrix**: TP / FP / TN / FN for classification.
- **Recall** = $TP / (TP + FN)$ — of all real positives, how many did we catch?
- **Precision** = $TP / (TP + FP)$ — of all predicted positives, how many were right?
- Trade-off: spam filter — high recall catches more spam (risk more false positives); high precision avoids blocking good mail.
- Confidence threshold: higher → more precision / less recall; lower → opposite.
- **Feature importance**: which inputs drive predictions (helps feature selection).

Model serving

- **Online / endpoint**: real-time, low latency, on-demand.
- **Batch prediction**: no endpoint needed; run when immediate response isn't required.
- **Edge / off-cloud**: when latency, privacy, or offline use matter (e.g. plant camera detection).
- After deploy: monitor continuously; Agent Platform Pipelines can automate alerts when metrics cross thresholds.

MLOps & pipelines

- MLOps = ML development + operations (DevOps ideas applied to models).
- Goal: continuous integration (CI), continuous training (CT), continuous delivery (CD) — because data and code both change.
- Agent Platform Pipelines supports Kubeflow Pipelines (KFP) and TensorFlow Extended (TFX). Prefer TFX if already deep in TensorFlow at scale; else KFP.
- Pipeline = ordered components (like functions). Use pre-built Google components or write custom ones.
- Automation maturity: Phase 0 GUI only → Phase 1 automate pieces → Phase 2 full CI/CT/CD pipeline.
- Build pattern: plan components → compile pipeline → submit PipelineJob (templates exist for AutoML tabular classification/regression).

How a neural net learns (optional essentials)

- Families: ANN → DNN / CNN / RNN / LLMs for different problems.
- ANN structure: input layer → hidden layer(s) → output layer; links have **weights** (what training learns).
- Forward step: weighted sum (+ bias) → **activation function** → next layer.
- Learning = adjust weights so predictions get closer to truth (loss ↓) over epochs.

Extra — AI Hypercomputer

- Integrated system for training/serving heavy AI with strong price-performance.
- Layer 1: GPUs/TPUs + fast network + optimized storage.
- Layer 2: open software (PyTorch, GKE, Kueue, etc.).
- Layer 3: flexible buying (on-demand, spot, CUDs, reservations, DWS).

- Deploy options: GCE (max control) → GKE → toolkits → Vertex AI / managed (easiest, less control).
- Use cases: large-scale training, efficient serving, app development on open frameworks.

Quick reference — pick the right tool

- Forecast/classify with existing tabular data in BigQuery → BigQuery ML.
- Need a custom model quickly, little coding → Agent Platform AutoML.
- Common perception task, no labeled data → pre-trained API.
- Unique architecture / research control → custom training.
- Chat / create / multimodal / agents → Gemini + Agent Studio / Enterprise / ADK.
- Always: prepare good data → train/evaluate → deploy/monitor → automate with MLOps.

Sources: M0_Course_introduction, M1_AI_foundations, M2_Generative_AI, M3_AI_development_options, M4_AI_development_workflow, M5_Summary, T-AIHYPE-l-m2-l1-en-file-8.en.pdf